UNIVERSITAT POLITÈCNICA DE CATALUNYA BARCELONATECH

FIB

PROP Project - Hidato

Team 12.4

version 3.0

Aghzaf El Hachimi, Mohamed	(mohamed.aghzaf)
Cifuentes Salamí, Félix	(felix.cifuentes)
Miquel Mitjans, Rafel	(rafel.miquel)
Ollers Rigo, Joan Pere	(joan.pere.ollers)

Table of contents

1. Hidato generation....4

Test - Generating different valid hidatos....4

2. Hidato resolution & validation....5

Test 1- Solving valid hidatos....5

Test 2 - Solving invalid hidatos....5

Test 3- Validating already resolved hidatos....6

3. Interactive game....8

Test 1- Correct Game Completion....8

Test 2 - Incorrect Game Completion....9

Test 3- Topology and Adjacency Consistency....9

4. Propose & validate user's hidatos....11

Test 1- Board Creation and Proposal....11

Test 2 - Validation of Solvability....12

Test 3- Machine Solving....13

5. Hidato repository....14

Test 1- Repository Access....14

Test 2 - Shared State Update....15

Test 3- Repository Navigation and Selection....16

6. Time control....17

Test 1- Control during a game session....17

Test 2 - Save and load times....17

7. View Ranking....18

Test 1- Creating Users, playing ranked mode and looking the ranking....18

8. Save a game session and reload it....19

Test 1- Create and save a Game Session....19

Test 2- Load a game session without logging out....20

Test 3- Logging a game session after logging out and logging in....21

Test 4- Logging a game session after closing and restarting the application....21

Test 5- Creating another game Session.....22
Test 6- Creating a new casual game but exiting without saving.....22
9. Select Difficulty and Get hints..... 23
Test 1- Solving valid hidatos.....23

1. Hidato generation

Test - Generating different valid hidatos

Test object

- The studied use cases are: GenerateHidato.

Data files needed

- None. This test is made using the generate hidato tool on the app.

Studied values

- To make sure the hidato generation works properly for any possible input, we tested the number of cell values: 2, 3, 20, 100, 200, 201, for different values of topology, adjacency and difficulty.
  We did not try every single combination for all numbers of cells, since that would mean doing over 90 different tests. However, every value was tested, and every combination was tested for 100 cells.

Studied effects

- What was studied was the ability to generate solvable hidatos for each value. The hidatos being solvable was checked by using the machine solve button. It was also studied that limits on the number of cells work properly for each case. It was also studied that by generating with the same settings multiple times you don't get the same hidato.

2. Hidato resolution & validation

Test 1- Solving valid hidatos

Test object

- The studied use cases are: MachineSolve

Data files needed

- In the folder /T2.1 there are several valid hidatos used in this test. Simply load them from the app Create/Import Hidato -> Select File and then press Play Hidato and the Machine Solve button.

Studied values

- The studied values are the different topologies, adjacencies, difficulties and number of cells. Since it's impossible to try every possibility with every number of cells, the used hidatos' attributes are chosen randomly making sure that every value is tested (for number of cells this means a change between small numbers of cells and large ones).

Studied effects

- It is checked that every valid hidato can be solved by the app, giving a correct solution in a reasonably short time.

Test 2 - Solving invalid hidatos

Test object

- The studied use cases are: MachineSolve

Data files needed

- In the folder /T2.2 there are several invalid hidatos used in this test. These were made by manually modifying the previous valid hidatos to make them unsolvable. Simply load them from the app Create/Import Hidato -> Select File and then press Play Hidato and the Machine Solve button.

Studied values

- The studied values are the different topologies, adjacencies, difficulties and number of cells. Since it's impossible to try every possibility with every number of cells, the used hidatos' attributes are chosen randomly making sure that every value is tested (for number of cells this means a change between small numbers of cells and large ones).
  Hidatos are modified by switching random cell values so that it turns unsolvable, but maintaining a correct format so that it is readable.

Studied effects

- It is checked that the app successfully identifies that it is invalid and notifies the user with an error message.

Test 3- Validating already resolved hidatos

Test object

- The studied use cases are: ValidateHidato

Data files needed

- Since you can only validate hidatos after you solve them, it is tested by manually solving hidatos correctly/incorrectly, meaning there is no

direct file to the test. However, the hidatos used for this are the ones from the folder /T2.1.

Studied values

- The studied values are the different topologies, adjacencies, difficulties and number of cells. Since it's impossible to try every possibility with every number of cells, the used hidatos' attributes are chosen randomly making sure that every value is tested (for number of cells this means a change between small numbers of cells and large ones).

Studied effects

- It is checked that the app correctly validates an hidato when completed, successfully notifying the user whether the given solution is correct or not.

3. Interactive game

Test 1- Correct Game Completion

Test object

- We are testing the game end logic of the game. The primary goal is to verify that the system correctly identifies when a user has successfully filled the entire board with a valid, consecutive sequence of numbers that obeys the established adjacency rules.

Data files needed

- We require game sessions that have almost complete boards and where only the final few cells are empty, as well as test boards that represent a fully completed, valid Hidato.

Studied values

- We study the transition from the "in-progress" state to the "solved" state. We test this across all the topologies (triangles, squares, and hexagons) and both types of adjacency (sides only and both sides and vertices) to ensure it works regardless of the game configuration. We do this for both casual and ranked games.

Studied effects

- The system immediately triggers the hidato complete pop up upon the placement of the final correct number. We observe that the application displays a confirmation message, stops the internal timer, and logs the final execution time as required for game records.

Test 2 - Incorrect Game Completion

Test object

- We are testing the system's ability to distinguish between a full board and a correctly completed board. The goal is to ensure the game prevents users from finishing a hidato if there are logical errors, even if every cell is occupied.

Data files needed

- We use game sessions where all cells are filled, but the numerical sequence contains errors, such as non-consecutive numbers, broken paths, or violations of the selected adjacency type.

Studied values

- We test scenarios where the board is fully populated but logically invalid. We study how the validator handles these cases to ensure it does not mistakenly flag an incorrect board as a win. We test in all topologies and adjacencies.

Studied effects

- The application correctly identifies that the board is full but it's not a correct solution and in all cases the 'incorrect solution' pop up appears.

Test 3- Topology and Adjacency Consistency

Test object

- We are testing the game's ability to adapt its rule-enforcement engine to different board geometries and adjacency definitions.

Data files needed

- We use a variety of Hidato boards with differing topologies (triangles, squares, and hexagons) and configured with different adjacency constraints (sides only and sides and vertices).

Studied values

- We study whether the game correctly identifies adjacent neighbors for each specific cell type, as the rules for connectivity change significantly between a square grid and a hexagonal or triangular one.

Studied effects

- We observe that the system correctly maps the neighbors for every cell according to the selected topology and adjacency rules. This ensures that when the final validation happens, the adjacency check is correct for that specific board type.

4. Propose & validate user's hidatos

Test 1- Board Creation and Proposal

Test object

- We are testing the 'Propose' feature (in our case we named it Create/Import Hidato), which allows a user to design their own custom Hidato puzzle from scratch. The main goal is to verify that the application can correctly capture and store the user's definition of the board's topology, the type of cells used (triangles, squares, or hexagons), the specific placement of "hole" cells, and the initial pre-filled numbers provided by the user as clues.

Data files needed

- We require various input strings representing different board designs, ranging from small, simple grids to large, complex topologies. These files must include the definition of the board's shape, its adjacency type, the locations of the holes and the list of initial numbers to be placed by the user.

Studied values

- We study the correctness of the input system, specifically how it handles different grid sizes and various configurations of starting numbers. We also test the boundary conditions of the proposal system, such as defining boards with no starting numbers.

Studied effects

- We observe that the internal representation of the board perfectly matches the user's intended design. The system correctly creates the

board and that all holes are correctly flagged so that they remain inaccessible during the game. It is worth mentioning that trivial board creations are also allowed, for example all cells are holes or there is a single cell, etc.

Test 2 - Validation of Solvability

Test object

- We are testing the "Validate" functionality. The goal is to determine if the machine can perform a thorough logical analysis to confirm that a user proposed board has at least one valid solution.

Data files needed

- We need different types of hidatos, for each combination of topology (triangles, squares and hexagons), each type of adjacency type (only sides or both sides and vertices) and incorrect ones.

Studied values

- We study the performance of the validation engine. We test if the engine can quickly distinguish between a valid, solvable grid and an impossible one, and we measure the time taken to validate these different puzzles.

Studied effects

- We verify that if there is a solution the solver fills the board accurately, demonstrating a correct, unbroken path from the lowest number to the highest, and confirming that the solution provided is fully compatible with the board's specific topology and adjacency rules. Otherwise a pop up informs that no valid solution exists.

Test 3- Machine Solving

Test object

- We are testing the "Resolve" feature, which uses the machine's internal solving algorithms to compute the complete sequence for a user's Hidato. The goal is to prove that the solver can handle user-generated boards.

Data files needed

- We are studying the solving of different types of hidatos, for each combination of topology (triangles, squares and hexagons), each type of adjacency type (only sides or both sides and vertices) and incorrect ones.

Studied values

- We look at the correctness of the solver, checking if it works on different board topologies and varying degrees of difficulty. We specifically test if the solver can successfully navigate through empty boards that have very few starting clues, which are typically the most difficult to solve.

Studied effects

- The machine correctly identifies if it can or can't be solved. And when it can be solved it does so correctly and relatively quickly. We note that it struggles the most with triangular boards with both kinds of adjacencies, since the number of adjacent cells to check is a lot higher than in other cases.

5. Hidato repository

Test 1- Repository Access

Test object

- We are testing the *LoadRepository* feature. The main goal is to make sure the game can look in the storage folder, spot all the available Hidato puzzles, and show them to the player in a list so they can pick one.

Data files needed

- We need two types of files: an index file that keeps track of the games, plus a few sample .hidato files like easy_hidato.csv, difficult_hidato.csv, etc. These files hold all the data: the board shape, where the "holes" are, and the starting numbers.

Studied values

- We are studying the loading of different types of hidatos, for each combination of topology (triangles, squares and hexagons) and each type of adjacency type (only sides or both sides and vertices). We also check hidatos saved in different stages: just after creating it, after filling some of the cells and after completing it.

Studied effects

- We have made sure in all studied hidatos the loading is done correctly, and the hidato is playable and correct. It is indeed completely empty and otherwise identical.

Test 2 - Shared State Update

Test object

- We are testing the integrity of the saveMatch logic when dealing with files from a shared repository. The goal is to ensure that a user's progress is recorded in their own personal directory (the match file is different from the hidato itself), keeping the original repository template as read-only.

Data files needed

- We use the hidatos saved to a repository (and not random ones) and a generated progress save-file that contains the user's specific cell data and timestamps for his specific match.

Studied values

- We check for different hidatos that once a user loads it and begins a match if he saves his match only he can access it and the original file for the hidato is unchanged. We check this for different hidatos at different stages and for different users.

Studied effects

- The repository master file remains the same. When the user reloads the session, we verify that every previously placed number appears exactly where it should, and that the timer resumes from the correct point, confirming the correctness of the saved state.

Test 3- Repository Navigation and Selection

Test object

- We are testing the navigation requirement, specifically the system's ability to display an organized, user-friendly interface that lists available Hidato puzzles from the repository. The goal is to verify that the player can accurately browse, filter, and select a specific puzzle from a set of files and also create new lists/repositories.

Data files needed

- We require several different lists/directories of Hidatos and several Hidato files (in .csv format). We need empty lists, lists with many Hidatos and different kinds of Hidatos.

Studied values

- We study the usability of the listing system when the repository contains a lot of files, when it is empty, when we move in and add an Hidato, when we remove an Hidato and finally, when we select and choose an Hidato to play.

Studied effects

- We observe that the list generation is correct and shows all files currently stored on disk. We confirm that clicking a specific entry in the list reliably launches the intended game session with the correct board. We can also properly add and remove both lists and Hidatos. You are also correctly stopped from creating two lists with the same name.

6. Time control

Test 1- Control during a game session

Test object

- The studied use cases are: PauseTimer, ResumeTimer, StartTimer.

Data files needed

- In the folder /T6 there are several started gamesessions and their hidatos to try. You can copy both folders into your data directory if you want to try them. However, this test does not only rely on them, since it is also checked for in-game generated game sessions.

Studied values

- For this test the studied values are each difficulty when loading/generating a match in both casual and ranked gamemodes.

Studied effects

- It is checked that the timer can be successfully started for playing and paused/resumed while playing. Also, that it still works properly after loading a game session from storage.

Test 2 - Save and load times

Test object

- The studied use cases are: LoadMatch, SaveMatch.

Data files needed

- In the folder /T6 there are several started gamesessions and their hidatos to try. You can copy both folders into your data directory if you want to try them. However, this test does not only rely on them, since it is also checked for in-game generated and non-started game sessions.

Studied values

- For this test the studied values are each difficulty when loading/generating a match in both casual and ranked gamemodes.

Studied effects

- It is checked that stored game sessions keep track of their timer and storing/loading a game session pauses/resumes the timer correctly.

7. View Ranking

Test 1- Creating Users, playing ranked mode and looking the ranking

Test object

- The studied use cases are: view Ranking

Data files needed

- The users User1,...,User5 (with passwords 1,...,5 respectively), their game sessions and the hidatos created. This users, game sessions and hidatos can be found in the folder /T7/users, /T7/gamesessions and /T7/hidatos respectively

Studied values

- 5 different users have been created. Many games in ranked mode have been played with each of the user accounts. Different difficulty levels have been played and games have been finished on several stages of the game (one incorrect cell, all correct values but the hidato not finished, all cells filled and all values correct, hints used, …).
- The main objective of these tests have been to prove the correct update of the ranking score of each user according to the solution delivered.

Studied effects

- When the user hands over an incorrect or incomplete solution the ranking score of the user is correctly maintained (no score is deducted).
- When the user delivers a correct answer their ranking score is updated correctly according to the difficulty level, hints used and time spent solving it.
- When the user spends too much time solving the hidato, or the number of hints used is too big the score is correctly unmodified (no score is added nor deducted).

8. Save a game session and reload it

Test 1- Create and save a Game Session

Test object

- The studied use cases are: Save Match, Load Match

Data files needed

- None

Studied values

- The creation of a user and the subsequent login of this user into the app, creation of a casual match and save of the match. In this test a user named UserSaveCasual has been created, with password "1234". Immediately after this action the user has logged in, created a casual match, introduced some correct and incorrect values and saved the match (The user, game session and hidato created can be found in the folder /T8.1).

Studied effects

- The game session is correctly saved and appears with its identifier on the list of game sessions of the user.

Test 2- Load a game session without logging out

Test object

- The studied use cases are: Save Match, Load Match

Data files needed

- The user and game session created by the previous test

Studied values

- Once a game session has been saved, without logging out the user loads the game session and fills in some more cells. After these changes the user saves and exits the game.

Studied effects

- The game session is correctly updated and appears with its identifier on the list of game sessions of the user.

Test 3- Logging a game session after logging out and logging in

Test object

- The studied use cases are: Save Match, Load Match

Data files needed

- The user and game session created by the previous test

Studied values

- After logging out, the user logs in and loads the match again. When finished editing the match saves it and closes the application.

Studied effects

- The game session is correctly updated and appears with its identifier on the list of game sessions of the user.

Test 4- Logging a game session after closing and restarting the application

Test object

- The studied use cases are: Save Match, Load Match

Data files needed

- The user and game session created by the previous test

Studied values

- The user restarts the application, logs in and loads the saved match.

Studied effects

- The game session has been correctly loaded and appears with its identifier on the list of game sessions of the user.

Test 5- Creating another game Session

Test object

- The studied use cases are: Save Match, Load Match

Data files needed

- The user and game session created by the previous test

Studied values

- The user restarts the application, logs in and creates another casual game session, fills in some cells and saves the current state (The new hidato and game session can be found in the folder /T8.5).

Studied effects

- The game session has been correctly saved and it does not create conflict with the already existing matches of the user.

Test 6- Creating a new casual game but exiting without saving

Test object

- The studied use cases are: Save Match, Load Match

Data files needed

- None

Studied values

- The user UserSaveCasual logs in, starts a new casual game session, fills in some values and exits the view without saving the changes.

Studied effects

- The game session has not been created due to the early exit and the previous game sessions remain untouched.

9. Select Difficulty and Get hints

Test 1- Select Easy difficulty on Ranked Mode

Test object

- The studied use cases are: PlayRanking

Data files needed

- None

Studied values

- 6 hidatos were created to play in ranked mode with the difficulty set as easy (The resulting hidatos can be found in the folder T9.1).

Studied effects

- All the created hidatos obey the ~25% of empty cells ratio.

Test 2- Select Medium difficulty on Ranked Mode

Test object

- The studied use cases are: PlayRanking

Data files needed

- None

Studied values

- 6 hidatos were created to play in ranked mode with the difficulty set as medium (The resulting hidatos can be found in the folder T9.2).

Studied effects

- All the created hidatos obey the ~50% of empty cells ratio.

Test 3- Select Hard difficulty on Ranked Mode

Test object

- The studied use cases are: PlayRanking

Data files needed

- None

Studied values

- 6 hidatos were created to play in ranked mode with the difficulty set as hard (The resulting hidatos can be found in the folder T9.3).

Studied effects

- All the created hidatos obey the ~75% of empty cells ratio.

Test 4- Select Easy difficulty on Casual Mode

Test object

- The studied use cases are: PlayCasual

Data files needed

- None

Studied values

- 6 hidatos were created to play in casual mode with the difficulty set as easy (The resulting hidatos can be found in the folder T9.4).

Studied effects

- All the created hidatos obey the ~25% of empty cells ratio.

Test 5- Select Medium difficulty on Casual Mode

Test object

- The studied use cases are: PlayCasual

Data files needed

- None

Studied values

- 6 hidatos were created to play in casual mode with the difficulty set as medium (The resulting hidatos can be found in the folder T9.5).

Studied effects

- All the created hidatos obey the ~50% of empty cells ratio.

Test 6- Select Hard difficulty on Casual Mode

Test object

- The studied use cases are: PlayCasual

Data files needed

- None

Studied values

- 6 hidatos were created to play in casual mode with the difficulty set as hard (The resulting hidatos can be found in the folder T9.6).

Studied effects

- All the created hidatos obey the ~75% of empty cells ratio.

Test 7- Get hint with the board almost empty

Test object

- The studied use cases are: Get Hint

Data files needed

- None

Studied values

- An hidato was created and immediately after the creation 5 hints were asked (The resulting game session can be found in the folder T9.7).

Studied effects

- 5 cells of the hidato were correctly filled.

Test 8- Get Hint with the board wrongly completed

Test object

- The studied use cases are: Get Hint

Data files needed

- None

Studied values

- An hidato was created and after wrongly solving the hidato a hint was asked (The resulting game session can be found in the folder T9.8).

Studied effects

- The app correctly informs the user that no hint is available due to it being completely solved.